Introduction to AI Project Problem Descriptions
Spring 2025 – 2026

General Note to Students:

Data Collection Bonus: For all projects, teams are strongly encouraged to collect local, real-world data (e.g., from ENSIA, a local hospital, Algiers public transport, or local companies).

- **Standard Grade:** Using the suggested international benchmark datasets.
- **Bonus Grade (+15%):** Successfully collecting, cleaning, and using a local dataset to solve the problem for a real local stakeholder.

==========

Project 1: Intelligent University Exam Scheduling System

Aim: Develop an AI-driven scheduling system that automates the generation of university exam timetables. The system must assign exams to time slots and rooms while satisfying strict logical constraints (hard constraints) and optimizing for student comfort (soft constraints) using Search Techniques (Greedy, A*, Genetic Algorithms) and Constraint Satisfaction Problems (CSP).

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Students are highly encouraged to collect real anonymized data from ENSIA or a department at some Algerian university. This includes:
    - **Course Catalog:** A list of offered courses (e.g., CS101, MATH202) with the total number of enrolled students.
    - **Student Enrollment Log:** A raw list or CSV file mapping Student IDs to the Courses they are taking (e.g., Student_123: [CS101, MATH202]). **Note:** This raw data is required to *compute* the conflict constraints.
    - **Room Inventory:** A list of available rooms with their specific seating capacities (e.g., Amphi 1: 200, Room 10: 30).
  - **Standard (International):** Use the **Toronto Examination Scheduling Benchmarks (Carter, 1996)**, a standard dataset for academic timetabling research.
- **External Resources:**
  - Research the "Graph Coloring Problem" and its application to timetabling.
  - Investigate "weighted penalty" systems used in academic scheduling to quantify student inconvenience.

b. Problem Definition:

The objective is to assign a valid Time Slot and Room to every Course Exam. The schedule must be conflict-free (feasible) and compact, minimizing the number of days students must attend while ensuring no student has overlapping exams.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard (Mandatory):** No student can sit for two exams at the same time. Room capacity must exceed the number of students. No two exams can be in the same room at the same time.
  - **Soft (Desirable):** Minimize the number of students with consecutive exams (e.g., three in 24 hours). Avoid late evening exams if possible.

- **Objective Function:**
    - **Minimize Penalty Cost**.
    - **Maximize Utilization:** Efficient use of room capacity (minimizing empty seats but with reasonable spacing, leaving at least one empty seat between any two students on the same row).

d. Search Strategy Implementation:

- **Greedy Search:**
    - Implement a "Degree Heuristic" (schedule the most conflicting exams first) or "Largest Enrollment First".
    - Assign the most difficult exam to the first available non-conflicting slot.
- **Constraint Satisfaction Problem (CSP):**
    - Model the problem with Variables (Exams), Domains (Time/Room pairs), and Constraints (Clashes).
    - Implement Backtracking Search with Minimum Remaining Values (MRV) and Forward Checking to prune invalid branches early.
- **Genetic Algorithms (GA):**
    - Evolve a population of schedules. Use a fitness function that penalizes hard constraint violations heavily and soft constraints lightly.
    - Implement crossover operators that swap "weeks" of schedules between *parents*.
- *A* Search:*
    - Treat the schedule as a state space. The heuristic *h(n)* estimates the "difficulty" of scheduling the remaining unassigned exams based on their conflict density.

e. Comparative Evaluation:

- **Performance Comparison:**
    - Compare the execution time of CSP vs. GA for small vs. large datasets.
    - Compare the quality (Penalty Cost) of the schedule produced by Greedy vs. A*.
- **Success Criteria:**
    - The system must produce a valid schedule (0 hard constraint violations) for the test dataset.
    - The interface must clearly highlight any unavoidable soft constraint violations.

f. Deliverables:

- **Working Prototype:** A desktop/web app that accepts CSV inputs (Courses, Students, Rooms) and outputs a valid timetable.
- **Visualizations:** A "Master Schedule" grid view and a "Student Conflict Heatmap" showing the most stressful days.
- **Documentation:** Detailed explanation of the heuristic used and a comparative analysis of algorithm performance.

Project 2: The Intelligent Hospital Nurse Scheduling

Aim: Create an automated monthly shift schedule for hospital nurses that strictly adheres to labor laws and ensures fair distribution of night and weekend shifts, using CSP and Local Search techniques.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Interview a head nurse or administrator at a local (private or public) reasonably sizeable hospital to gather real constraints (e.g., "Senior nurses must be on night shifts", "Legal rest time is 12 hours"). Create a dataset of 20-50 staff members with specific qualifications.
  - **Standard (International):** Use the *Nottingham Nurse Rostering Benchmarks* or similar datasets from the "Employee Scheduling Benchmarks" repository.
- **External Resources:**
  - Research local labor laws regarding maximum working hours and mandatory rest periods.
  - Study "Cyclic Rostering" vs. "Preference-based Rostering".

b. Problem Definition:

The objective is to assign a Shift Type (Day, Night, Late, Off) to every Nurse for every Day of the planning period (e.g., 30 days). The schedule must ensure sufficient coverage for patient safety while respecting staff well-being.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** Max consecutive working days (e.g., 5). Minimum rest time between shifts (e.g., cannot do Night → Day). Required skill mix (at least one Senior Nurse per shift).
  - **Soft:** Honor specific "Day Off" requests. Equalize the total number of night shifts among all nurses.
- **Objective Function:**
  - **Minimize Violation Score:** Sum of weighted penalties for denied vacation requests and unequal shift distribution.
  - **Maximize Fairness:** Minimize the variance in total hours worked across staff.

d. Search Strategy Implementation:

- **Constraint Satisfaction Problem (CSP):**
  - Define constraints formally. Use propagation techniques to ensure hard constraints (legal laws) are never broken during search.
- **Simulated Annealing (Local Search):**
  - Start with a random (potentially invalid) schedule. "Swap" shifts between nurses to reduce the violation score. Accept worse solutions with a decreasing probability to escape local optima.
- **Tabu Search:**
  - Similar to Simulated Annealing but maintains a "Tabu List" of recently swapped shifts to prevent cycling back to previous bad states.
- **Greedy Search:**
  - Assign shifts day-by-day, prioritizing the nurse with the "least hours worked so far."

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the convergence speed of Simulated Annealing vs. Tabu Search.
  - Analyze how the "Greedy" approach fails to satisfy long-term constraints (like monthly hour limits) compared to the global view of CSP.
- **Success Criteria:**
  - The final roster (planning) must be legally valid.
  - The system should demonstrate a "fairness metric" improvement over a random assignment.

f. Deliverables:

- **Working Prototype:** An interactive calendar interface allowing manual overrides of the AI-generated schedule.
- **Visualizations:** Color-coded roster view and "Fairness Charts" (bar charts showing hours worked per nurse).
- **Documentation:** Formal definition of the cost function and a report on the effectiveness of Local Search for this problem.

==========

Project 3: The "Smart Diet" Meal Planner

Aim: Develop an AI-based meal planning system that generates a 30-day diet plan meeting strict nutritional goals and budget constraints, using Genetic Algorithms and CSP.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Scrape or manually collect food prices from a local supermarket (e.g., online grocery store) and build a database of local recipes with their nutritional breakdown.
  - **Standard (International):** Use the USDA FoodData Central database for nutrition and generic pricing datasets.
- **External Resources:**
  - Research standard nutritional guidelines (WHO or local health authority) for macro-nutrient ratios (Carbs/Protein/Fat).
  - Investigate the "Knapsack Problem" and how it relates to budget optimization.

b. Problem Definition:

Select a sequence of meals (Breakfast, Lunch, Dinner) for 30 days. The plan must be nutritionally balanced, financially viable, and sufficiently varied to be edible.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** Daily Calorie count within $\pm 10\%$ of user TDEE (Total Daily Energy Expenditure). $Total\ cost \leq User\ Budget$.
  - **Soft:** Variety (do not repeat the same dinner more than twice a week). User preferences (e.g., "Vegetarian", "High Protein").
- **Objective Function:**
  - **Minimize Cost:** Find the cheapest combination of foods that meets nutrition goals.

- **Maximize Nutritional Accuracy:** Minimize the deviation from the target macro-nutrient split (e.g., 40% Protein, 30% Carbs, 30% Fat).

d. Search Strategy Implementation:

- **Genetic Algorithms (GA):**
  - **Chromosome:** A 30-day meal plan (90 meals).
  - **Mutation:** Randomly replace a "Lunch" with another "Lunch" from the database.
  - **Fitness:** A weighted score combining Cost, Nutritional Delta, and Variety.
- **Constraint Satisfaction Problem (CSP):**
  - Use **Arc Consistency (AC-3)** to filter the domain of possible meals for each day based on the remaining daily calorie allowance.
- **Greedy Search:**
  - Select the "best" meal for each slot sequentially based on the current nutrient deficit.
- *A* Search:*
  - Search for the optimal sequence of meals for *one week* (doing 30 days might be too deep for A*).

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the "Variety Score" of plans generated by GA vs. Greedy.
  - Evaluate computation time: Can CSP generate a valid day plan instantly?
- **Success Criteria:**
  - The system generates a plan that costs less than the budget while hitting nutritional targets.
  - The plan is diverse (low repetition rate).

f. Deliverables:

- **Working Prototype:** A simple app where users input their height, weight, goal, and budget, and get a PDF meal plan.
- **Visualizations:** Pie charts of macro-nutrient distribution and a "Cost vs. Nutrition" scatter plot.
- **Documentation:** Explanation of the fitness function and how specific dietary restrictions (e.g., allergies) were modeled as constraints.

==========

Project 4: The Sports League Scheduler

Aim: Design a system to schedule a complete season for a professional sports league, optimizing for fairness, travel distance, and broadcast revenue using Constraint Programming and Local Search.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Use the actual teams, stadium locations (GPS coordinates), and "Derby" (rivalry) lists from the *Algerian Ligue 1* (football) or Super Division (basketball).
  - **Standard (International):** Use a synthetic dataset of 16-20 teams with random coordinates.
- **External Resources:**
  - Study the "Traveling Tournament Problem" (TTP), a famous benchmark in Operations Research.

- Research "Home/Away Break Minimization".

b. Problem Definition:
Create a "Double Round-Robin" schedule where every team plays every other team twice (once at home, once away), assigned to specific "Rounds" (weeks).

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** Each team plays exactly one match per round. Every pair meets exactly twice.
  - **Soft:** No team plays more than 2 consecutive Away games. "Top Matches" (Derbies) should be scheduled on weekends or specific TV slots.
- **Objective Function:**
  - **Minimize Total Travel:** The sum of distances traveled by all teams throughout the season.
  - **Maximize Fairness:** Minimize the difference in "Rest Days" between opponents.

d. Search Strategy Implementation:

- **Constraint Satisfaction (CSP/CP):**
  - Model the schedule as a grid. Use specialized constraints (e.g., AllDifferent) to ensure valid pairings.
- **Hill Climbing (Local Search):**
  - Start with a standard "Canonical Schedule" (pattern-based). Randomly swap the slots of two matches and check if the Total Travel Distance decreases.
- **Simulated Annealing:**
  - Use SA to avoid getting stuck in a local optimum where the travel distance is "okay" but not "optimal."
- **Greedy Search:**
  - Construct the schedule round-by-round, always picking the match that adds the least travel distance for the current teams.

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the Total Travel Distance of the schedule produced by Hill Climbing vs. Simulated Annealing.
  - Determine the limit of your solver: Experiment with increasing league sizes (10, 14, 16, 18, 20 teams) and report at which size the algorithm fails to return a solution within a fixed time limit.
- **Success Criteria:**
  - A valid double round-robin schedule is produced.
  - The optimized schedule shows a measurable reduction in travel km compared to a random schedule.

f. Deliverables:

- **Working Prototype:** A web dashboard showing the season calendar and a specific "Team View" (their path).
- **Visualizations:** A map overlay showing the travel route of a specific team across the country.
- **Documentation:** Mathematical formulation of the Traveling Tournament Problem constraints.

Project 5: The "Green" Multi-Modal Public Transit Router

Aim: Develop a multi-modal routing engine that finds optimal paths between two points in a city by combining walking, bus, tram, and metro networks. The system must optimize for a weighted cost function of Time, Money, and Carbon Footprint (CO_2), using advanced Graph Search algorithms (A*, Dijkstra, Bi-Directional Search).

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Students are encouraged to digitize the public transport network of a specific district (e.g., Algiers Centre, Oran).
    - **Nodes:** Bus stops, Metro stations, Tram stops, and major intersections.
    - **Edges:** "Walk" (distance / 5km/h), "Ride Bus" (distance / avg_speed), "Wait" (frequency of service).
    - **Schedules:** Rough frequency data (e.g., "Metro every 5 mins").

    So we can consider that:
    - **Walk Edge:** Cost = Distance / 5.
    - **Ride Edge:** Cost = Distance / BusSpeed.
    - **Wait Edge:** Cost = Frequency / 2 (or a fixed penalty like 5 mins).
  - **Standard (International):** Use a standard *GTFS (General Transit Feed Specification)* dataset for a city like London, imported into a graph database or NetworkX.
- **External Resources:**
  - Research Multi-Modal Pathfinding and how to model "Transfer Penalties".
  - Investigate standard CO_2 emission factors for different transport modes (Bus vs. Metro vs. Car).

b. Problem Definition:

Given a Start Node (*A*) and a Goal Node (*B*), find the sequence of edges (Walk/Ride) that minimizes the total cost function. The graph is weighted and directed.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** Connectivity (can only switch from Bus to Metro at a valid transfer station).
  - **Soft:** Minimize number of transfers. Avoid walking more than 1km total.
- **Objective Function:**
  - $\text{Cost(n)} = w_1 \times \text{Time} + w_2 \times \text{Price} + w_3 \times CO_2$
  - The user should be able to adjust the weights (e.g., "I care only about the environment" so High w_3).

d. Search Strategy Implementation:

- **Dijkstra's Algorithm:**
  - Implement as a baseline to guarantee the mathematically shortest path.
- *A* Search:*
  - Implement A* with a custom heuristic function *h(n)*.
  - *h(n)* could be the "Euclidean distance to goal / Max Speed of Metro".

- **Bi-Directional Search:**
    - Run two simultaneous searches: one forward from Start and one backward from Goal, meeting in the middle. This often drastically reduces the search space.

e. Comparative Evaluation:

- **Performance Comparison:**
    - Compare the number of nodes expanded by Dijkstra vs. A* vs. Bi-Directional Search.
    - Analyze the trade-off: Does A* find the *absolute* best path as often as Dijkstra when the heuristic is aggressive?

- **Success Criteria:**
    - The system accurately calculates the total travel time including wait times.
    - The "Green Route" (High CO_2 weight) is distinct from the "Fastest Route" (High Time weight).

f. Deliverables:

- **Working Prototype:** A map-based interface (using Leaflet or Google Maps API) where users click Start/End and see the route drawn.
- **Visualizations:** A chart comparing the CO_2 impact of the generated route vs. taking a private car.
- **Documentation:** Mathematical derivation of the heuristic function and proof of its admissibility.

==========

Project 6: The "Algiers in 24 Hours" Tourist Optimizer

Aim: Design a tour planning system for tourists with limited time. The system must select the optimal subset of city landmarks to visit and order them to maximize the "Total Interest Score" within a strict time budget, using Local Search and TSP variants.

a. Data Collection & Research:

- **Dataset Overview:**
    - **Bonus (Local Data):** Create a curated dataset of 20–30 landmarks in Algiers (e.g., Casbah, Maqam Echahid, Ketchaoua Mosque, Jardin d'Essai).
        - **Attributes:** GPS coordinates, Opening Hours, "Visit Duration" (e.g., 1 hour), and "Interest Score" (1-10).
    - **Standard (International):** Use a standard *Orienteering Problem* benchmark dataset or a set of 50 random points with assigned scores.

- **External Resources:**
    - Study the *Traveling Salesperson Problem (TSP)* and the *Orienteering Problem* (TSP with profits/scores).
    - Research Simulated Annealing cooling schedules.

b. Problem Definition:

Find a path that starts and ends at the user's Hotel. The path must visit a subset of locations such that the *Total Time (Travel + Visit)* $\leq T_{max}$ (e.g., 12 hours) and the sum of Interest Scores is maximized.

c. Constraints & Objective Function:

- **Constraints:**
    - **Hard:** *Total Duration* $\leq$ *User's Time Budget*. Must start/end at Hotel. Must visit locations during their opening hours (Time Windows).

- **Objective Function:**
  - Maximize $\sum(\text{Interest\_Score}_i)$ for all visited locations *i*.

d. Search Strategy Implementation:

- **Greedy Search (Nearest Neighbor):**
  - From the current location, go to the nearest unvisited location with the highest score. Use this as a baseline.
- **Simulated Annealing (Local Search):**
  - Start with a random valid tour. Apply "operators" (Swap two stops, Remove a stop, Add a stop).
  - Accept worse solutions with probability $P = e^{-\Delta E/T}$ to escape local optima.
- **Genetic Algorithms (GA):**
  - Evolve a population of tours. Use "Order Crossover" (OX1) to preserve the relative order of visits while mixing parents.

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the "Total Score Collected" by Greedy vs. SA vs. GA.
  - Analyze how the solution quality degrades as the Time Budget becomes tighter (6 hours vs 12 hours).
- **Success Criteria:**
  - The route is physically possible (respects travel times).
  - The AI suggests a significantly better itinerary than a simple "closest first" approach.

f. Deliverables:

- **Working Prototype:** A "Day Planner" app showing the itinerary on a map with arrival/departure times.
- **Visualizations:** A convergence plot for Simulated Annealing showing how the "Total Score" improved over iterations.
- **Documentation:** Detailed explanation of the "Cooling Schedule" used in SA and the mutation rates in GA.

==========

Project 7: Emergency Ambulance Dispatch & Routing

Aim: Develop a dynamic dispatch system that assigns ambulances to incoming emergency calls and routes them through traffic to the nearest hospital, using Real-Time A* and Heuristic Clustering.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Map the exact locations of hospitals and ambulance depots in your city.
    - **Traffic Model:** Create a simple model where main roads (Highways) have variable speeds depending on the time of day (Rush Hour vs. Night).
  - **Standard (International):** Use a grid-based simulation where "Emergencies" appear randomly according to a Poisson distribution.
- **External Resources:**
  - Research Voronoi Diagrams for facility location/coverage.

- Study Dynamic Pathfinding (D* or Real-Time A*).

b. Problem Definition:
A continuous simulation where emergencies appear at coordinates (x, y) at time *t*. The system must:

1. Assign the best available ambulance.
2. Calculate the fastest path to the scene (Goal 1).
3. Calculate the fastest path from scene to hospital (Goal 2).

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** Ambulance capacity = 1 patient. Must go to Hospital before taking a new job.
  - **Soft:** Minimize "Response Time" (Time form Call to Arrival).
- **Objective Function:**
  - Minimize $\sum(\text{Response\_Time}_i)$ for all emergencies.
  - Maximize coverage (keep ambulances spread out when idle).

d. Search Strategy Implementation:

- Real-Time A*:
  - Pathfinding that updates as traffic weights change (e.g., if an edge becomes "blocked" or "slow", recalculate the path).
- Hill Climbing:
  - Goal: Find the optimal "Standby Locations" for ambulances when they are idle.
  - Method: Start with ambulances at random intersections. Calculate the "Average Response Time" to all historical accident points.
  - Think how to iterate.
- Greedy Assignment:
  - Simply assign the Euclidean-closest ambulance. Compare this to a smarter "Time-based" assignment using A*.

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the Average Response Time of "Static Stationing" (ambulances return to base) vs. "Dynamic Stationing" (ambulances move to high-risk areas).
  - Evaluate how the system handles a "surge" (many accidents at once).
- **Success Criteria:**
  - Visual demonstration of ambulances routing around "traffic jams" (high-weight edges).

f. Deliverables:

- **Working Prototype:** A simulation dashboard showing moving ambulances, pending emergencies, and hospitals.
- **Visualizations:** A "Response Time Histogram" and a "Traffic Heatmap."
- **Documentation:** Analysis of the traffic model and the dispatch logic.

Project 8: The Electric Vehicle Road Trip Planner

Aim: Create a long-distance route planner for Electric Vehicles (EVs) that eliminates "Range Anxiety" by intelligently scheduling charging stops, using Constrained A* and Breadth-First Search.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Use OpenChargeMap API to extract real charging station locations for Algeria (or North Africa).
    - **Vehicle Data:** Define specs for a specific car (e.g., Hyundai Ioniq 5: Range 480km, Battery 77.4kWh).
  - **Standard (International):** Use a dense dataset of charging stations from the US or Europe.
- **External Resources:**
  - Research the Shortest Path Problem with Resource Constraints (SPPRC).
  - Study battery consumption models (impact of speed and elevation).

b. Problem Definition:

Find a path from City A to City B. The state is defined not just by location, but by (Location, Current Battery %).

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** *Battery level* $> 0\%$ at all times.
  - **Soft:** Prefer "Fast Chargers" over "Slow Chargers." Minimize total trip duration (Driving Time + Charging Time).
- **Objective Function:**
  - Minimize $\text{TotalTime} = \sum(\text{Drive\_Time}) + \sum(\text{Charge\_Time})$.

d. Search Strategy Implementation:

- Breadth-First Search (BFS):
  - Explore all possible paths layer by layer. This will likely fail or run out of memory but serves as a baseline for "completeness."
- *Constrained A*:*
  - Augment the state: $\text{State} = (\text{Node}, \text{BatteryLevel})$.
  - Heuristic $h(n) = \textit{EuclideanDist}/\text{AvgSpeed}$.
  - Logic: If $\text{Battery} < \text{DistToNextNode}$, that edge is infinite cost (blocked).
- Greedy Best-First Search:
  - Always drive towards the goal until battery is low, then drive to nearest charger. (Show how this can get stuck in a "dead end").

e. Comparative Evaluation:

- **Performance Comparison:**
  - Show a scenario where Greedy fails while A* succeeds.
  - Compare the computation time of BFS vs. A*.
- **Success Criteria:**
  - The system successfully plans a trip that exceeds the car's single-charge range (e.g., 600km trip for a 480km range car).

f. Deliverables:

- **Working Prototype:** A map interface showing the route and explicitly marking "Charging Stops" with estimated charge times.
- **Visualizations:** A "Battery Profile" graph (Y-axis = Battery %, X-axis = Distance) showing the charge draining and refilling along the route.
- **Documentation:** Explanation of how the "Charging Time" was calculated in the cost function.

==========

Project 9: Automated Warehouse Robot Controller

Aim: Design a multi-agent control system for a fleet of warehouse robots (like Kiva/Amazon Robotics). The system must coordinate multiple robots to move shelves to packing stations without collisions or deadlocks, using Multi-Agent Pathfinding (MAPF) and Cooperative Search techniques.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Visit a local logistics center or large warehouse (e.g., Uno Hypermarket storage, Yalidine, or a university library archive). Sketch their floor plan to create a custom grid map.
    - **layout:** Create a grid file representing obstacles (shelves) and open space.
    - **Task List:** Generate a "Pick List" based on real observations (e.g., "Items from Aisle 3 are ordered most frequently").
  - **Standard (International):** Use the *MovingAI Benchmarks*, specifically the warehouse maps.
- **External Resources:**
  - Research *Multi-Agent Pathfinding (MAPF)* and the concept of "Reservation Tables" (Time-Space A*).
  - Study "Lifelong MAPF" where tasks appear continuously.

b. Problem Definition:

Given *N* robots at start positions and *N* goal positions (shelves to retrieve), find a set of conflict-free paths such that all robots reach their goals in minimal time.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** No two robots can occupy the same cell at the same time *(t)*. No "Edge Conflicts" (Robot A moves u $\rightarrow$ v while Robot B moves v $\rightarrow$ u).
  - **Soft:** Maintain a safety distance. Minimize "waiting" (idling).
- **Objective Function:**
  - **Minimize Makespan:** The time until the *last* robot finishes its task.
  - **Minimize Flowtime:** The sum of time taken by *all* robots.

d. Search Strategy Implementation:

- *Cooperative A* (Hierarchical):*
  - Plan paths sequentially. Robot 1 plans first (ignoring others). Robot 2 plans next, treating Robot 1's path as moving obstacles (dynamic barriers).
- *Independent A* (Baseline):*
  - Plan all robots independently. If a collision occurs, simple "wait and retry." (Expected to fail/deadlock in crowded maps).

- *Hill Climbing:*
    - Optimization phase: Take a valid solution and try to "shorten" paths by swapping move orders.

- *Conflict-Based Search (CBS) (Bonus):*
    - A two-level search. High level searches the "Conflict Tree" (resolving collisions), Low level finds paths for individual agents.

e. Comparative Evaluation:

- **Performance Comparison:**
    - Compare the success rate of *Independent A** (how often does it deadlock?) vs. *Cooperative A**.
    - Measure the "Total Steps" for varying fleet sizes (5 robots vs. 20 robots).

- **Success Criteria:**
    - Visual demonstration of 10+ robots moving simultaneously without crashing.
    - The system handles a "Head-on Collision" scenario gracefully (e.g. one robot moves aside).

f. Deliverables:

- **Working Prototype:** A grid-based simulation (GUI) showing robots as moving colored dots carrying shelves.
- **Visualizations:** A "Heatmap" of congestion (which corridors are most blocked?).
- **Documentation:** Detailed explanation of the collision-checking logic in the *Time* dimension.

==========

Project 10: The 3D Container Loading Optimizer

Aim: Develop a logic-based logistics tool that computes the optimal packing of 3D boxes into a standard shipping container to maximize volume utilization and stability, using Genetic Algorithms and Heuristic Search.

a. Data Collection & Research:

- **Dataset Overview:**
    - **Bonus (Local Data):** Obtain a real "Manifest" (list of packages) from a local courier (e.g., Yalidine, EMS, or a package moving company).
        - **Items:** Measure 20–50 real box types (dimensions L $\times$ W $\times$ H and weight).
        - **Container:** Use standard truck dimensions from the local company.
    - **Standard (International):** Use a synthetic dataset of 100 boxes with random dimensions and a standard ISO 20ft container.

- **External Resources:**
    - Research the *3D Bin Packing Problem (3D-BPP)*.
    - Study "Deepest Bottom-Left Fill" (DBLF) strategies.

b. Problem Definition:

Place a set of rectangular boxes $B = \{b_1, b_2, \dots\}$ into a container *C* such that they fit completely and do not overlap.

c. Constraints & Objective Function:

- **Constraints:**
    - **Geometric:** Box must be inside container boundaries. No overlap.
    - **Physical:** "Gravity Constraint" (Every box must be supported by the floor or another box). "Orientation" (Fragile boxes cannot be rotated upside down).

- **Objective Function:**
  - **Maximize Volume Utilization:** $\frac{\sum \text{Vol}(b_i)}{\text{Vol}(C)} \times 100\backslash\%$.
  - **Maximize Stability:** Heuristic to place heavy items at the bottom.

d. Search Strategy Implementation:

- *Genetic Algorithms (GA):*
  - **Chromosome:** A permutation (sequence) of boxes to pack.
  - **Decoder (Heuristic):** Take the sequence from the GA and place each box using a "Best Fit" logic.
  - **Fitness:** The total volume packed.
- *Greedy Heuristic (Best-Fit Decreasing):*
  - Sort boxes by volume (largest first). Place each box in the first available space that fits.
- *Simulated Annealing:*
  - Perturb the sequence of boxes (swap two boxes) and re-evaluate the packing efficiency.

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare the Volume % achieved by Greedy vs. GA. (Does the GA find a non-obvious combination?).
  - Analyze execution time of the three methods.
- **Success Criteria:**
  - The system generates a valid packing plan (no floating boxes).
  - Utilization exceeds 75-80% for standard box mixes.

f. Deliverables:

- **Working Prototype:** A desktop app where users input box dimensions and see the result.
- **Visualizations:** A 3D render of the container (using libraries like Three.js or Matplotlib) that allows rotating/zooming to see the stack.
- **Documentation:** Algorithm for the "Space Management" (how free space is tracked).

==========

Project 11: University Course-to-Room Allocator

Aim: Distinct from Exam Scheduling, this project focuses on the semester-long assignment of lectures to physical classrooms. The goal is to optimize space usage and minimize campus travel for students/faculty using Local Search.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Get the Room Inventory (Floor plans, capacity, type: Lab/Lecture) and Course Offerings (Expected attendance) from the ENSIA administration.
  - **Standard (International):** Use the *ITC2007 (International Timetabling Competition)* dataset track for "Curriculum-based Course Timetabling."
- **External Resources:**
  - Research *Simulated Annealing* and *Tabu Search* for resource allocation.
  - Study "Room Stability" (keeping a course in the same room all semester).

b. Problem Definition:

Assign a (Room, TimeSlot) tuple to every Course Event for the entire semester.

c. Constraints & Objective Function:

- **Constraints:**
  - **Hard:** *Room Capacity* ≥ *Course Enrollment*. Room Type match (Electronics in Lab). No double-booking of rooms.
  - **Soft:** Professor Preference (consecutive classes in same room), etc.
- **Objective Function:**
  - **Minimize Penalty:** The penalty will take into account the *Distance*, CapacityWaste, and RoomChange.

d. Search Strategy Implementation:

- *Simulated Annealing (SA):*
  - Start with a random valid allocation.
  - **Move Operator:** Select a course at random and move it to a different valid room.
  - **Temperature:** Start high (allow bad moves) and cool down (strict optimization).
- *Hill Climbing with Random Restart:*
  - Run Hill Climbing 50 times from different random starting points and pick the best result.
- *Constraint Satisfaction (CSP):*
  - Use CSP only to generate the *initial* valid solution (finding a feasible start point), then use SA to optimize it.

e. Comparative Evaluation:

- **Performance Comparison:**
  - Compare *Hill Climbing* vs. *Simulated Annealing*.
  - Show a graph of "Cost vs. Iterations."
- **Success Criteria:**
  - Zero Hard Constraint violations.
  - Demonstrable reduction in "wasted seats" compared to the current manual schedule used by the department.

f. Deliverables:

- **Working Prototype:** A room planning tool.
- **Visualizations:** A "Room Occupancy Heatmap" showing which rooms are empty/full. A "Professor Schedule" view.
- **Documentation:** Detailed report on the used cost function, any weights, and why.

==========

Project 12: The "Smart City" Waste Collection Router

Aim: Optimize the routing of garbage trucks to cover every street in a district while minimizing fuel consumption and shift time, using Genetic Algorithms and Arc Routing techniques.

a. Data Collection & Research:

- **Dataset Overview:**
  - **Bonus (Local Data):** Use OpenStreetMap (OSM) to export the street network of a local neighborhood (e.g., Hydra, Bab Ezzouar).
    - **Demand:** Simulate trash levels (kg) for each street segment based on population density (e.g., "Main street = 100kg", "Alley = 20kg").
  - **Standard (International):** Use *CARP (Capacitated Arc Routing Problem)* benchmark files (e.g., the standard egl or gdb datasets).

- **External Resources:**
    - Research the *Chinese Postman Problem (CPP)* and *Capacitated Arc Routing Problem (CARP)*.
    - Study *Eulerian Paths*.

b. Problem Definition:
Find a set of tours (routes) for K trucks starting and ending at the Depot, such that every edge with $demand > 0$ is traversed at least once.

c. Constraints & Objective Function:
- **Constraints:**
    - **Capacity:** *Total trash collected on a route* $\leq$ *Truck Max Load*.
    - **Time:** *Route duration* $\leq$ *Shift Length* (e.g., 8 hours).
    - **Topology:** Must follow one-way streets correctly.
- **Objective Function:**
    - **Minimize Total Distance:** The sum of lengths of all truck tours (including "deadheading" - traveling without collecting).
    - **Minimize Fleet Size:** Use fewer trucks if possible.

d. Search Strategy Implementation:
- *Genetic Algorithms (GA):*
    - **Chromosome:** A permutation of all required edges (streets).
    - **Split Procedure:** A heuristic that cuts the long chromosome into valid truck routes based on capacity.
- *Path Scanning (Greedy Heuristic):*
    - Construct routes one by one. At each intersection, choose the nearest unvisited street with high demand.
- *Local Search (2-Opt):*
    - Take a generated route and try to uncross loops (swap edges) to reduce distance.

e. Comparative Evaluation:
- **Performance Comparison:**
    - Compare the *Total Mileage* of the *Greedy approach* vs. the *Genetic Algorithm*.
    - Analyze how the solution scales: Does it work for 50 streets? 500 streets?
- **Success Criteria:**
    - 100% Coverage (no street left behind).
    - Visual proof of efficient routing (no trucks driving in circles).

f. Deliverables:
- **Working Prototype:** A map-based dashboard showing the district.
- **Visualizations:** Color-coded routes (Truck 1 = Red, Truck 2 = Blue). Animation of trucks clearing the streets.
- **Documentation:** Explanation of the "Split" algorithm used to convert the GA sequence into vehicle routes.